

Analysis: Cheating Detection

Solutions for this problem are based on the fact that the cheater's advantage pumps up their numbers independently of the difficulty. Judged by their total number of correct answers, a cheater with base skill level B is going to look like a player with skill $B + \Delta$ for some significant Δ (Δ depends on B). However, this cheater does worse on easy problems than a player of actual $B + \Delta$ skill level, and better on hard problems, because the correct answers that are coming from cheating happen uniformly instead of more heavily on easier problems like skill-based improvements.

Test Set 1

There are multiple ways to get to 10% accuracy to pass Test Set 1. One such way is to estimate each question's difficulty as its number of correct answers, then sort by that difficulty, and check how uniform the distribution of corrects for each candidate is. The closer to uniform a candidate looks, the more they look like a cheater. One possible metric is the number of pairs of questions with different answers such that the incorrect answer is estimated to be for an easier question than the correct one. Using this metric is just enough to pass Test Set 1.

Test Set 2

The issue with counting inversions as the suggested metric for Test Set 1 is that it is a metric that is very susceptible to the contestant's strength. More concretely, a list that has few corrects or few incorrects has fewer opportunities to have inversions than one that is fairly evenly split. We can solve this by dividing the number of inversions by the expected number of inversions in a randomly arranged one. This reduces the noise enough to pass Test Set 2.

Another way to increase accuracy, of inversions or any other metric, is to check super-easy and super-hard questions, because the difference between a uniform distribution of correct answers and a heavily biased distribution is more pronounced in those. Exactly how much depends on the metric, but around the easiest and hardest 5% of questions seems like the right number experimentally for our solutions.

Other metrics are more accurate than inversions and also help solve the problem. We found two techniques that work well enough: sort the players by estimated skill level (i.e., by total number of correct answers) and compare the number of corrects only in the "extreme" questions with the number of corrects of other players with similar estimated skill level among those. Then, assume that the cheater will have the greatest difference with its neighbors. Another technique could be to estimate the actual skill level (by computing the inverse sigmoid of the accuracy of each player) and the actual difficulty of questions (again, the inverse sigmoid of its proportion of corrects). Then, using those two, estimate the expected number of correct answers for each player in the extreme questions. The player with the largest difference between that estimation and the real value is the cheater. Using this latest technique can get a solution above 90% accuracy.

Analysis: Median Sort

This problem is about [information theory](#). There are $N!/2$ essentially different outputs, which means we need at least $\log_2(N!/2)$ bits of information. For $N = 50$ that is slightly over 213. That means that for Test Set 2, extracting less than a bit per query (on average) can work, but for Test Set 3 it will not. In Test Set 1, on the other hand, this informational analysis is not necessary.

Test Set 1

In this test set we have 300 queries per case, which is more than the number of different triples $\binom{N}{3} = 120$. This means that we can query every possible subset of 3 elements and memoize the results. Freed from the limitations of the number of queries, any solution to sort based on medians works. One simple way is to notice that the only elements that are never the median are the first and last elements, so we can identify those, and arbitrarily choose which one is the first and which one is the last. Then, we can eliminate those two and find the candidates for second and next-to-last as the ones that are never median in queries with only remaining elements. To know which one to assign second, we can check an additional query between the element we chose to go first and the two newfound elements: the one who is the median of that subset should be second, and the other one should be next-to-last. We can iterate this to find and place pairs of elements moving inwards until we place them all.

See below for other sorting algorithms that work on medians. While the other test sets have the additional burden of having to make them work in an [online](#) way, we can use the pre-query tactic for this one to make our lives easier, even if we implement the same basic algorithm. It also allows for simpler implementations of some of the algorithms that use extra queries.

Test Set 2

Since just 1 bit of information per query is enough for this test set, we can use one of our known comparison-based optimal [sorting algorithms](#) like Merge Sort or Heap Sort. Even [Insertion Sort](#) can work if we use [binary search](#) to look for the insertion point. While Insertion Sort + Binary Search requires a suboptimal $O(N^2)$ number of operations, it uses an optimal $O(N \log N)$ number of comparisons.

For any of these algorithms we need a way to simulate a binary "less than" operation between two arbitrary items i and j . One way to do that is to ask for the median of i , j and k while knowing k is not the median. So, we could try to find an overall minimum or maximum and then use that as k for every other query. Notice that there are 2 elements that can be minimum and/or maximum and neither would be the median of any query. So, we can do a query for the median of the first three elements. We can discard the median of those and keep the other 2 as candidates. We add any element we have not checked and do another median query, discarding the median and keeping 2 candidates again. After $N - 2$ queries, we have discarded $N - 2$ elements, and the 2 that remain are the minimum and maximum.

Test Set 3

To solve Test Set 3 we can observe that our implementation of Insertion Sort can actually extract more than 1 bit per query and have a smaller constant. Instead of looking for the minimum/maximum first, we simply use the minimum of our current range in the median query as k . This means it is not guaranteed to never be the median. However, if the "current minimum"

is the median of our query, we know exactly where to insert and can stop searching. This effectively makes our query a binary comparison with a little bit of extra information, and that little bit (plus being careful about never overspending) can be enough to pass Test Set 3. More importantly, it enables us to not find for the minimum first, which is a significant overexpenditure.

We can also find solutions that have more wiggle room by extracting a lot more than 1 bit per query. Specifically, we want to extract something closer to the optimal $\log_2 3 \approx 1.58$ bits per query. That means considering all 3 possible outcomes of the query, and have them happen with approximately equal probability (see [the information theory article](#) for details on the link between the probability distribution of outcomes and the information content of the query response).

We can further refine the Insertion Sort implementation to use a ternary search (in this case, this means a search similar to binary search that splits into three parts instead of two) to extract the full potential out of the median query. At each step, we query the two elements that are $1/3$ and $2/3$ of the way into our current range, together with the element to be inserted. The result narrows down the search to one of three ranges (roughly first, middle or last third). Notice that any query that we do to handle border cases (like inserting at the very beginning or very end) does not get the optimal 1.58 bits of information, so we want to be careful not to use those, or to use them infrequently.

A different option is to do a [two-pivot randomized quicksort](#). By using two pivots, we make full use of each median query involving them and each other element, as there are three buckets where the other elements can fall, each corresponding to a possible response to the query. The only issue with this approach (and any other approach based on a [divide and conquer](#) sorting algorithm) is that there are two ways to orient each recursive result if they have more than one element. If we use a query to decide which way is consistent with our decision on how to order the pivots, that query gives us only 1 bit of information. Luckily, this is rather infrequent as it only happens proportional to the number of branches in the recursion tree that contain more than one element, which is a small number.

Analysis: Moons and Umbrellas

Test Set 1

In Test Set 1, the size of the mural is small enough that we can test every possible final mural. Let ℓ be the length of S . We can either check for each string of length ℓ consisting only on C s and J s whether it agrees with S in the non-? spots, or brute-force only the ?s directly. For each valid way of finishing the mural, we can calculate the total cost to Cody-Jamal, and keep a running minimum to output at the end. Since we check at most 2^ℓ final murals, and for each we need only an additional linear pass to check the score, this algorithm takes $O(2^\ell \cdot \ell)$ time, which is fast enough for Test Set 1.

Test Set 2

In Test Set 2, ℓ can be up to 1000, so an exponential algorithm will not do. However, a simple observation can yield a much faster algorithm: since $X > 0$ and $Y > 0$, we would like to avoid inserting CJ or JC into the string. So if all letters surround a consecutive substring of ?s are the same letter a , then making all those ?s into a adds no CJ s or JC s. If the substring is surrounded by a on the left and b on the right, with $a \neq b$, on the other hand, at some point there will be a change from a to b , so we will be forced into an occurrence of ab . Making all the ?s into as (or bs) makes sure that we only add that one forced occurrence and nothing else. This yields a greedy algorithm that can be implemented in multiple ways, some taking as little as $O(\ell)$ time. Notice that the cost is the same on the resulting string than on the string where ?s are removed instead of replaced, which leads to a 1-line implementation:

```
S.replace('?', '').count('CJ') * X + S.replace('?', '').count('JC') * Y
```

Extra credit: Test Set 3

In the solution for Test Set 2 we assume that minimizing the number of occurrences of CJ and/or JC is best, which is true only when X and Y are non-negative. This means that Test Set 3 needs a completely different solution. In this case, a technique that would not normally show up in the second problem in a Qualification Round: [dynamic programming](#).

We can take the function $f(s)$ that we need to compute and define it recursively as follows:

- $f(?s) = \min(f(Cs), f(Js))$
- $f(a?s) = \min(f(aCs), f(aJs))$ for $a \in \{C, J\}$
- $f(aas) = f(as)$ for $a \in \{C, J\}$
- $f(CJs) = X + f(Js)$
- $f(JCs) = Y + f(Cs)$
- $f(s) = 0$ if the length of $s \leq 1$

You can verify that the cases above form a partition and properly define the recursion. We have a constant number of cases, each of which can be computed in constant time not counting the recursion calls, so the time complexity of the solution is $O(D)$ where D is the size of the domain of the function. You can check that every value of f that is ever required to calculate $f(S)$ can be defined by a suffix of the input S and at most two extra characters at the beginning. This means D is linear in the length of S , and so is the time complexity of the algorithm. Notice that we need to represent each element in the domain in constant space for this to work out exactly

as mentioned, for example representing suffixes of S by just their length or an index into S . However, for the low limits of this problem, even a larger representation using copies of the full suffix is fast enough.

Analysis: Reversort

The pseudocode of the solution is almost given in the statement. We just need to give it the formal shape in the programming language of our choice. There are two non-straightforward statements in the provided pseudocode. We assume we store L in an array, so we have quick access to any index within it.

The first one is to figure out the index of the minimum element in a contiguous subarray. There may be some library functions to perform this task. For example, in C++ we can use [`min\_element`](#), in Python we can use [`index`](#) and [`min`](#) to figure out the minimum element. We can also run another loop to find out the minimum element in the subarray. Please note, the input numbers are all different so the minimum in each iteration is unique.

The second one is to reverse a subarray. Again, there may be some library functions to perform this task. For example, we can use the [`reverse`](#) STL library function for C++, [`reversed`](#) or [`reverse`](#) or simply slicing in Python. We can also run a loop to reverse the subarray.

The length of the subarray we are reversing in the second step above is the cost of the reversal. Accumulating these costs will give our final answer.

This solution has the time complexity of $O(N^2)$. We are running an outer loop from 1 to $N - 1$. Inside the loop we are performing two steps that take linear time each: minimum finding and array reversal. Hence the time complexity is $O(N^2)$.

One final note, there are solutions to this problem that run in $O(N \log N)$ time, but they are a lot more complex. Do you want to try to find one?

Analysis: Reversort Engineering

Test Set 1

The solution to the problem is a permutation of the numbers from 1 to N . The cost of each permutation can be calculated by simulating the Reversort algorithm as described in the [analysis of the Reversort problem](#) in $O(N^2)$ time complexity. There are $N!$ distinct permutations of size N , containing the numbers from 1 to N exactly once each. The cost of each permutation can be calculated and the answer is any permutation that has a cost equal to C . If there is no such permutation, output `IMPOSSIBLE`. The time complexity of the overall solution is $O(N! \cdot N^2)$.

Test set 2

As N is large for Test Set 2, we cannot generate every possible permutation. The major observation here is that the range of valid costs for a given N lies between $N - 1$ (when the cost of each reverse operation is the minimum possible, which is 1) and $\frac{N \cdot (N+1)}{2} - 1$ (when the cost of each reverse operation is the maximum possible, which is $N - i$). Cost = $N - 1$ when the array is already sorted.

All costs in between those two limits are possible, as we shall see. Hence, if C is not in the valid range for given N , output `IMPOSSIBLE`. Otherwise, we perform the following construction by recursion, which also serves as proof that the costs in range are indeed possible. The first iteration costs between 1 and N , so we should choose a cost x for it such that $C - x$, fits in the possible range for a permutation of size $N - 1$. You can check that this is always possible, and even compute the full range of x values that work by solving the system of inequalities.

Now, recursively generate a permutation P of size $N - 1$ and cost $C - x$. Then, add 1 to all integers in P and insert 1 at its left end, getting a new permutation of integers between 1 and N . Then, reverse the prefix of P of length x as the cost of the initial iteration should be x . The non-recursive steps take $O(N)$ to adjust P . Since we perform those for each index, the overall complexity of the solution is $O(N^2)$.